



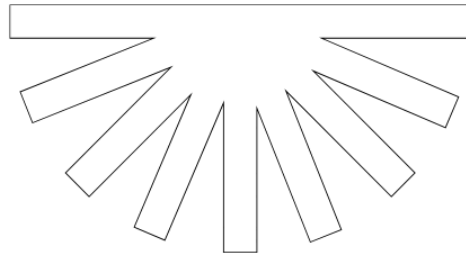
TECNOLÓGICO
NACIONAL DE MÉXICO

TECNOLÓGICO NACIONAL
DE MÉXICO
INSTITUTO TECNOLÓGICO
DE HERMOSILLO



GYM TRACKER

PROYECTO FINAL



PROGRAMACIÓN WEB

MOROYOQUI IBARRA JONATHAN ADRIAN

UNIDAD 5

Fernández Sánchez Alexandra
Palafox Morales Valeria
Ochoa Fuentes Luis Felipe
Porras Estrella Joseph Steven

25/Mayo/2026

Ing. Sistemas Computacionales S6A

Introducción

Descripción general del sistema

Gym Tracker es una aplicación web desarrollada con ASP.NET Core en el backend y React en el frontend, diseñada para que cualquier persona que entrena de forma independiente pueda organizar y registrar sus entrenamientos de manera sencilla y eficiente. El sistema permite gestionar un catálogo de ejercicios, crear rutinas personalizadas, iniciar sesiones de entrenamiento en tiempo real y consultar el historial de sesiones anteriores.

Problema que resuelve

Muchas personas que asisten al gimnasio de forma independiente no tienen una manera estructurada de llevar el seguimiento de sus entrenamientos. Dependen de notas en el celular, hojas de papel o simplemente la memoria para recordar qué ejercicios hicieron, cuántas series completaron o qué peso levantaron. Esto dificulta medir el progreso y mantener una rutina consistente a lo largo del tiempo.

Objetivo general

Desarrollar una aplicación web funcional que permita a usuarios independientes gestionar sus rutinas de entrenamiento, registrar sus sesiones en tiempo real con series, repeticiones y pesos, y consultar su historial de entrenamientos, todo desde una interfaz intuitiva y accesible.

Documentación técnica

Tecnologías utilizadas

Backend:

.NET 10.0.203 — ASP.NET Core Web API
Entity Framework Core 10.0.0 — ORM Code First
BCrypt.Net-Next 4.2.0 — hash de contraseñas
Swashbuckle.AspNetCore 10.1.7 — documentación Swagger
Microsoft SQL Server 2025 — base de datos

Frontend:

React 19.2.6
React Router DOM 7.15.1
Vite 8.0.12

Arquitectura

El proyecto sigue una arquitectura cliente-servidor. El backend es una Web API REST desarrollada en ASP.NET Core que expone endpoints HTTP consumidos por el frontend. La comunicación entre ambas capas se realiza mediante peticiones fetch en React hacia la API, intercambiando datos en formato JSON. El backend se conecta a SQL Server a través de Entity Framework Core usando el patrón Code First, donde los modelos en C# definen la estructura de la base de datos.

La estructura del proyecto backend se organiza en carpetas: Models para las entidades, Data para el contexto de base de datos, Controllers para los endpoints, y Dtos para objetos de transferencia. El frontend React se encuentra dentro de la carpeta vista/.

Base de datos

Entidades y atributos principales:

Usuario — representa a cada persona registrada en el sistema.

- UsuarioId (int, PK), Nombre (string), Email (string), PasswordHash (string), FechaRegistro (DateTime)

Ejercicio — catálogo global de ejercicios disponibles.

- EjercicioId (int, PK), Nombre (string), Descripcion (string), GrupoMuscular (string), Dificultad (string)

Rutina — rutina de entrenamiento creada por un usuario.

- RutinaId (int, PK), UsuarioId (int, FK), Nombre (string), Descripcion (string), FechaCreacion (DateTime)

RutinaEjercicio — tabla intermedia que relaciona rutinas con ejercicios.

- RutinaEjercicioId (int, PK), RutinaId (int, FK), EjercicioId (int, FK), Series (int), Repeticiones (int), Orden (int)

Sesion — registro de un entrenamiento iniciado por un usuario.

- SesionId (int, PK), UsuarioId (int, FK), RutinaId (int, FK), FechaInicio (DateTime), FechaFin (DateTime, nullable)

SesionDetalle — detalle de cada ejercicio realizado dentro de una sesión.

- DetalleId (int, PK), SesionId (int, FK), EjercicioId (int, FK), SeriesHechas (int), RepsHechas (int), PesoKg (decimal)

Relaciones entre tablas:

- Un Usuario puede tener muchas Rutinas
- Una Rutina pertenece a un Usuario
- Una Rutina puede tener muchos Ejercicios a través de RutinaEjercicio
- Un Ejercicio puede pertenecer a muchas Rutinas a través de RutinaEjercicio
- Un Usuario puede tener muchas Sesiones
- Una Sesion pertenece a un Usuario y a una Rutina
- Una Sesion puede tener muchos SesionDetalles
- Un SesionDetalle pertenece a una Sesion y referencia un Ejercicio

API

Usuarios

POST	/api/usuarios/registro	→ Registrar nuevo usuario
POST	/api/usuarios/login	→ Autenticar usuario
GET	/api/usuarios/{id}	→ Obtener perfil de usuario
PUT	/api/usuarios/{id}	→ Actualizar datos de usuario
DELETE	/api/usuarios/{id}	→ Eliminar usuario

Ejercicios

GET	/api/ejercicios	→ Obtener todos los ejercicios
GET	/api/ejercicios/{id}	→ Obtener un ejercicio por ID
POST	/api/ejercicios	→ Crear nuevo ejercicio
PUT	/api/ejercicios/{id}	→ Actualizar ejercicio
DELETE	/api/ejercicios/{id}	→ Eliminar ejercicio

Rutinas

GET /api/rutinas → Obtener todas las rutinas
GET /api/rutinas/{id} → Obtener rutina con sus ejercicios
POST /api/rutinas → Crear nueva rutina
PUT /api/rutinas/{id} → Actualizar rutina
DELETE /api/rutinas/{id} → Eliminar rutina
POST /api/rutinas/{id}/ejercicios → Agregar ejercicio a rutina
DELETE /api/rutinas/{id}/ejercicios/{ejercicioId} → Quitar ejercicio de rutina

Sesiones

POST /api/sesiones → Iniciar entrenamiento
PUT /api/sesiones/{id}/finalizar → Finalizar entrenamiento
POST /api/sesiones/{id}/detalle → Registrar ejercicio realizado
GET /api/sesiones/usuario/{usuarioId} → Obtener historial del usuario
GET /api/sesiones/{id} → Obtener sesión con detalle completo

Distribución del trabajo

Integrante	Backend	Frontend
Fernández Sánchez Alexandra	Configuración inicial del proyecto ASP.NET Core, modelo Usuario, DbContext, AppDbContext, endpoints de registro y login con hash de contraseña (BCrypt), validación de email, migraciones y configuración de base de datos SQL Server	—
Ochoa Fuentes Luis Felipe	Modelos Ejercicio, Rutina y RutinaEjercicio, EjerciciosController con CRUD completo, RutinasController con endpoints de creación, edición y asociación de ejercicios a rutinas, seed de ejercicios predefinidos	—
Palafox Morales Valeria	Modelos Sesion y SesionDetalle, SesionesController con endpoints para iniciar, finalizar y registrar detalles de entrenamiento, así como consulta de historial por usuario	Configuración del proyecto React con Vite, React Router, componente de navegación, vista de Login y Registro
Porras Estrella Joseph Steven	—	Dashboard, Catálogo de Ejercicios, Mis Rutinas, Iniciar Entrenamiento (Módulo 1), Historial de Entrenamientos (Módulo 2)

Retroalimentación del proyecto

Fernández Sánchez Alexandra

¿Qué fue lo más difícil de desarrollar y cómo lo resolviste? Lo más difícil fue configurar correctamente la base de datos con Entity Framework Core, especialmente las relaciones entre entidades. Al momento de aplicar las migraciones, SQL Server rechazaba la creación de la tabla Sesiones. Lo resolví configurando manualmente las relaciones en el método OnModelCreating del DbContext.

¿Qué herramientas o recursos externos consultaste? Usé Swagger como herramienta para probar los endpoints directamente desde el navegador.

¿Qué cambiarías o mejorarías si tuvieras más tiempo? Implementaría autenticación para que el sistema sea más seguro, en lugar de manejar la sesión solo con localStorage.

¿Qué aprendiste que no sabías antes de iniciar el proyecto? Aprendí a configurar una Web API desde cero con .NET, a trabajar con Entity Framework Core en Code First y a entender cómo funciona el flujo completo entre la base de datos, el backend y el frontend.

Ochoa Fuentes Luis Felipe

¿Qué fue lo más difícil de desarrollar y cómo lo resolviste? Lo más difícil fue implementar correctamente el controlador de rutinas, especialmente los endpoints para agregar y quitar ejercicios de una rutina.

¿Qué herramientas o recursos externos consultaste? Revisé ejemplos en GitHub de proyectos similares con ASP.NET Core Web API.

¿Qué cambiarías o mejorarías si tuvieras más tiempo? Implementaría filtros por grupo muscular directamente en el backend en lugar de hacerlo en el frontend.

¿Qué aprendiste que no sabías antes de iniciar el proyecto? Aprendí a construir una API REST completa con .NET, a manejar relaciones complejas entre tablas usando Entity Framework Core y a estructurar un proyecto backend de forma organizada siguiendo buenas prácticas.

Palafox Morales Valeria

¿Qué fue lo más difícil de desarrollar y cómo lo resolviste? Lo más difícil fue conectar el frontend con el backend correctamente, ya que el navegador bloqueaba las peticiones por la política de CORS. Lo resolví verificando que el backend tuviera configurada la política de CORS apuntando al puerto correcto de React, y asegurándome de que el servidor estuviera corriendo al momento de probar el frontend.

¿Qué herramientas o recursos externos consultaste? Revisé ejemplos de fetch en JavaScript para consumir APIs REST desde React.

¿Qué cambiarías o mejorarías si tuvieras más tiempo? Le daría un diseño más pulido y responsivo a todas las vistas.

¿Qué aprendiste que no sabías antes de iniciar el proyecto? Aprendí a crear un proyecto React con Vite desde cero, a configurar React Router para la navegación entre páginas y a consumir una API REST desde el frontend usando fetch con métodos GET y POST.

Porras Estrella Joseph Steven

¿Qué fue lo más difícil de desarrollar y cómo lo resolviste? Lo más difícil fue centralizar el estado de la aplicación para lograr que múltiples componentes aislados se comunicaran entre sí en tiempo real por ejemplo al pasar datos de una rutina creada a la pantalla de entrenamiento, y de ahí empaquetar las series acumuladas hacia el historial.

¿Qué herramientas o recursos externos consultaste? La pestaña Console, que fue vital para rastrear excepciones críticas en el renderizado. También gitHub Desktop: Utilizado como interfaz visual para gestionar las ramas del repositorio y resolver los conflictos de código surgidos por trabajar en los mismos archivos simultáneamente en equipo.

¿Qué cambiarías o mejorarías si tuvieras más tiempo? Si la aplicación local o la api de c# se desconectan, los datos vuelven a su estado semilla inicial al recargar la página. Implementaría localStorage o una base de datos robusta para asegurar que las rutinas y el historial de entrenamientos no se borren nunca.

¿Qué aprendiste que no sabías antes de iniciar el proyecto? Aprendí a interpretar y resolver de forma segura los conflictos de mezcla cuando dos desarrolladores editan las mismas líneas de código, decidiendo estratégicamente qué versión debe predominar en la rama principal sin destruir el progreso del equipo y que JavaScript puede ser sumamente estricto con los parámetros de configuración, y que un solo carácter erróneo puede congelar la ejecución completa de un botón en toda la aplicación.

Retroalimentación del curso

Integrante 1:

¿Qué temas del curso fueron más útiles para el proyecto? Todos los temas vistos en el curso fueron de utilidad directa para el proyecto, el profesor los fue desarrollando a través de un proyecto guiado que cubría exactamente los mismos requisitos que se pedían en el proyecto final. Eso facilitó mucho la comprensión y aplicación de cada tema.

¿Qué temas sentiste que necesitaban más práctica o explicación? En general el curso requería bastante esfuerzo para seguir el ritmo, principalmente porque el trabajo se proyectaba al frente del salón y no siempre se veía con claridad desde todos los lugares. Sin embargo, algo que se valora mucho es que el profesor pasaba por cada lugar revisando que todos tuvieran el avance correcto y resolviendo dudas y errores de forma personalizada.

¿Qué le agregarías o quitarías al curso? Agregaría prácticas más pequeñas por tema a manera de tareas, para reforzar de forma individual cada concepto visto en clase antes de avanzar al siguiente.

¿Cómo calificarías el curso del 1 al 10 y por qué? 9. El profesor es muy abierto a las dudas, explica con disposición y demuestra dominio del tema. Quizás le falta un poco más de experiencia frente al grupo, pero en general la clase fue muy buena.

Comentarios adicionales para el profesor: Ninguno.

Integrante 2:

¿Qué temas del curso fueron más útiles para el proyecto? Los temas más útiles fueron React, el consumo de APIs y la conexión entre frontend y backend, porque fueron fundamentales para que el sistema funcionara correctamente.

¿Qué temas sentiste que necesitaban más práctica o explicación? Algunos temas de React y la comunicación con la API necesitaban más práctica, especialmente para quienes no teníamos mucha experiencia previa.

¿Qué le agregarías o quitarías al curso? Agregaría más ejercicios pequeños por tema para reforzar mejor los conceptos antes del proyecto final.

¿Cómo calificarías el curso del 1 al 10 y por qué? 8. Fue un curso útil y se logró desarrollar un proyecto funcional. El profesor apoyaba resolviendo dudas y revisando avances durante las clases.

Comentarios adicionales para el profesor: Gracias por la disposición para ayudar durante el semestre y por apoyar al grupo en el desarrollo del proyecto.

Integrante 3:

¿Qué temas del curso fueron más útiles para el proyecto? Pienso que todos los temas que vimos son y serán útiles, pero los que siento que fueron más útiles, los de las APIs (y APIs Axios) para los datos, y cuando separamos la lógica con los slices y actions para no tener todo revuelto.

¿Qué temas sentiste que necesitaban más práctica o explicación? igual las APIs y lo del SQL siento que este último necesitó un poco más de profundidad, también con lo del react ya que si no tienes mucho conocimiento si se puede llegar a complicar un poco en seguir el curso.

¿Qué le agregarías o quitarías al curso? Le agregaría que en vez de que sea una práctica larga que pueda llegar a durar 2 semanas, que sean ejercicios pequeños que puedan ser por clase, ya que así siento que aprenderías mejor y más probable que todos puedan seguir el ritmo del curso.

¿Cómo calificarías el curso del 1 al 10 y por qué? Un 9, el maestro a pesar que a veces se iba muy rápido y no alcanzabas a leer en el proyector, siempre estaba dispuesto a ayudarte hasta que corrigieras los errores que pudieran salir durante la clase, y me gusta que el maestro es paciente con los alumnos.

Comentarios adicionales para el profesor: Gracias por tenernos paciencia y enseñarnos un poco de lo mucho que sabe.

Integrante 4:

¿Qué temas del curso fueron más útiles para el proyecto? El manejo del estado global fue la espinosa dorsal del proyecto. Entender cómo conectarse al componente padre en nuestra App.jsx que nos permitió comunicar las rutinas con el entrenamiento activo y el historial sin perder información en el camino; También el consumo de APIs ya que ayudó a conectar nuestra interfaz de React con el servidor backend local de C# con el puerto 5050 para traer dinámicamente los datos del perfil de usuario.

¿Qué temas sentiste que necesitaban más práctica o explicación? Tuvimos problemas serios al usar useNavigate fuera de un contexto válido de router, lo que nos congelaba la pantalla. Creo que hace falta profundizar en cuándo conviene usar un enrutador formal y cuándo es mejor manejar pestañas por estado local.

¿Qué le agregarías o quitarías al curso? Le agregaría más prácticas en clase, más contexto de para qué es cada cosa que utilizamos e ir al tiempo del salón en general y así reforzar cada punto que vimos en clase, desde el Html hasta la Api.

¿Cómo calificarías el curso del 1 al 10 y por qué? 8, mediante las clases hubo momentos en los que no sabía que estaba haciendo ni para qué, y para alguien que nunca ha visto javascript y quiere aprender, fue desanimador, pero ya con el proyecto y gracias al trabajo en equipo, pude entender un poco lo que vimos a lo largo del curso, aprendimos cosas 100% funcionales y eso me gusto!.

Conclusiones

El desarrollo de Gym Tracker representó un reto importante para el equipo, ya que implicó integrar tecnologías que estábamos aprendiendo al mismo tiempo que las aplicábamos. A lo largo del proyecto construimos una aplicación web funcional que cubre desde la gestión de usuarios y catálogos hasta el registro en tiempo real de entrenamientos y la consulta de historial, cumpliendo con los requisitos establecidos desde el inicio.

¿Logramos el objetivo planteado?

Sí. El objetivo general era desarrollar una aplicación que permitiera a usuarios independientes gestionar sus rutinas, registrar sus sesiones de entrenamiento y consultar su historial, y ese objetivo se cumplió. El sistema cuenta con un backend robusto desarrollado en ASP.NET Core con Entity Framework Core conectado a SQL Server, y un frontend en React que consume la API y presenta la información de forma clara e intuitiva. Además se implementaron los dos módulos funcionales requeridos: iniciar un entrenamiento activo y consultar el historial de sesiones pasadas.

El proyecto nos enseñó que la comunicación entre los integrantes es tan importante como el código mismo. Dividir el trabajo por áreas — backend y frontend — nos permitió avanzar en paralelo, pero también nos hizo ver la importancia de definir bien los contratos entre ambas partes desde el inicio, como los nombres exactos de los endpoints y el formato de los datos.